Technical Requirements Document (TRD)

Overview

One Python backend, one React front end, one PostgreSQL database and one separate simulated NHIS service, all started with `docker compose up`. Every coding agent follows this document so the six modules fit together without rework.

Technical goals

- **Runs anywhere with one command:** a reviewer clones the repo, adds an API key, runs Docker Compose, and sees the demo.
- **One way to do each thing:** one LLM layer, one database, one masking function, one event mechanism, shared by all modules.
- **Every AI output is checked:** schema validation, guardrails and logging on every LLM call.
- **Measurable:** every module has an eval script whose results feed the dashboard.

Technology stack

Area	Tool	Purpose
Backend	Python 3.11, FastAPI, Pydantic v2	API, validation, auto-generated API docs at `/docs`
Database	PostgreSQL 16 + pgvector	All records, plus vector search for RAG
Database access	SQLAlchemy 2 + Alembic	Models and migrations
LLM	Anthropic SDK (Claude, default); OpenAI and Gemini SDKs behind the same interface	Extraction, RAG answers, rewriting, intent, agent, claim checks
Embeddings	sentence-transformers, `BAAI/bge-small-en-v1.5` (local)	RAG vectors; no data leaves the system
ML	pandas, scikit-learn, xgboost, shap, joblib	Module 3 no-show model
PDF	pymupdf	Module 2 document loading

Area	Tool	Purpose
Text checks	textstat	Module 4 readability
Scheduling	APScheduler	Daily reminder and scoring jobs
Front end	React 18 + Vite + TypeScript, Tailwind CSS	All screens
Live updates	WebSockets (fallback: polling every 2 seconds)	Patient chat, alerts
Simulated NHIS	Separate small FastAPI app in `sim_nhis/`	Independent claim adjudication
Observability	Structured JSON logs; Langfuse optional	Traces, cost, latency
Packaging	Docker Compose	`api`, `web`, `db`, `sim_nhis` services
CI	GitHub Actions	Tests on every push; evals on demand

System architecture

- **`api`** (FastAPI) holds all six modules as separate packages: `scribe`, `knowledge`, `risk`, `messenger`, `coordinator`, `claims`, plus shared `core` (LLM layer, masking, audit, events, config).
- **`web`** (React) calls only `api`.
- **`db`** is shared; each module owns its own tables (see Backend Schema).
- **`sim_nhis`** is a separate service that `claims` calls over HTTP. It must not import any `api` code.
- **Modules talk through function calls inside `api` and one internal event: `note.approved`.** With `COORDINATOR_ENABLED=true` the coordinator handles it; with `false`, risk, messenger and claims handle it directly.

Shared services every module must use

Service	Location	Rule
LLM layer	`core/llm.py`	One function `complete(task, messages, tools=None, schema=None)`; temperature 0; timeout and retries built in; logs model, prompt version, tokens, latency
Prompts	`prompts/*.txt` with version in the file name	Never inline prompts in code
Masking	`core/masking.py`	Any patient text sent to an LLM goes through `mask()` first
Audit log	`core/audit.py`	Every AI action and every human approval is written to `audit_log`
Config	`core/config.py` reading `.env`	No hard-coded keys, model names or thresholds

API conventions

- REST, JSON, prefixes by module: `/notes`, `/kb`, `/risk`, `/messenger`, `/coordinator`, `/claims`, `/admin`.
- Errors: `{ "error": { "code": "...", "message": "..." } }` with proper HTTP status.
- The current role comes from an `X-Role` header set by the role switcher (no real login in this version).

Non-functional requirements

Area	Requirement
Security	API keys only in `.env` (never committed); masking before every LLM call; role checks on every endpoint
Safety	Human approval before any record, message or claim becomes final

Area	Requirement
Performance	Note draft < 30 s; RAG answer < 5 s; care plan < 30 s; claim check < 15 s
Reliability	LLM failure never blocks a user; every module has a manual fallback
Reproducibility	All generators and training seeded (42); `make reset-demo` rebuilds all data
Testing	Unit tests for every guardrail; one end-to-end test of Ama's journey

Environment variables

Variable	Example	Used by
`ANTHROPIC_API_KEY`	(secret)	LLM layer
`OPENAI_API_KEY`, `GEMINI_API_KEY`	(optional)	Provider comparison
`LLM_DEFAULT_MODEL`	a current Claude model name	LLM layer
`DATABASE_URL`	`postgresql://mediflow:mediflow@db:5432/mediflow`	All
`COORDINATOR_ENABLED`	`true`	Event routing
`RAG_MIN_SIMILARITY`	`0.50`	Module 2
`SIM_NHIS_URL`	`http://sim_nhis:8001`	Module 6
`DEMO_MODE`	`true`	Shows role switcher and demo controls